

Predictive Water Pump Activation for Smart Irrigation System

Project Description

Data related to IoT-based environmental monitoring and control. The data encompasses various sensor readings and actuator states, aimed at automating and optimising environmental conditions. The goal is to explore patterns, trends, and anomalies within the dataset, providing insights that can enhance the efficiency and effectiveness of environmental control systems. By leveraging this data, the project aims to develop predictive models and decision-making tools that can improve the management and sustainability of environmental resources.

Scenario 1: Automated Greenhouse Management

In a controlled greenhouse environment, the IoT sensors continuously monitor various environmental factors. By analysing this data, the system can predict the best times to activate fans, water pumps, and other actuators to maintain optimal growing conditions. This scenario focuses on enhancing crop yield and reducing resource waste by automating responses to real-time environmental changes.

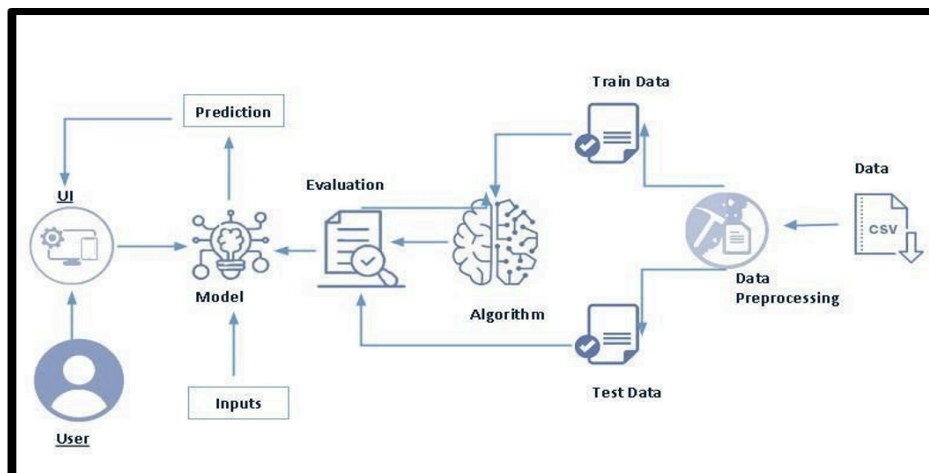
Scenario 2: Smart Irrigation System

An agricultural field is equipped with IoT sensors to monitor soil moisture, temperature, and humidity levels. The data collected is used to create a predictive model that determines the precise amount of water needed for different sections of the field. The smart irrigation system can then automatically activate the watering pumps, ensuring efficient water usage and promoting sustainable farming practices.

Scenario 3: Environmental Anomaly Detection

In an urban area, IoT devices are deployed to monitor environmental parameters such as air quality, temperature, and humidity. By analysing historical data and real-time sensor readings, the system can detect anomalies or unusual patterns that may indicate potential environmental hazards. This scenario aims to provide early warnings and actionable insights to city officials and residents, helping to mitigate the impact of environmental issues and improve public health and safety.

Technical Architecture:



Project Flow:

- User interacts with the UI to enter the input.
- Entered input is analysed by the model which is integrated.
- Once model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below:

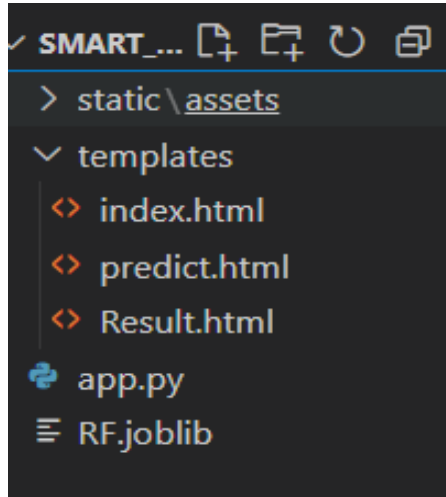
- **Data Collection & Preparation**
 - Collect the dataset
 - Data Preparation
- **Exploratory Data Analysis**
 - Descriptive statistical
 - Visual Analysis
 - Balancing
- **Model Building**
 - Training the model in multiple algorithms
 - Testing the model
- **Performance Testing**
 - Testing model with multiple evaluation metrics
 - Comparing model accuracy before & after applying hyperparameter tuning
- **Model Deployment**
 - Save the best model
 - Integrate with Web Framework

Prior Knowledge:

You must have the prior knowledge of the following topics to complete this project.

- ML Concepts:
- Supervised learning: <https://www.javatpoint.com/supervised-machine-learning>
- Logistic Regression: <https://www.geeksforgeeks.org/understanding-logistic-regression/>
- Gradient boosting regressor: <https://www.geeksforgeeks.org/ml-gradient-boosting/>
- Decision Tree Classifier: <https://www.geeksforgeeks.org/decision-tree/>
- Flask Basics: https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Structure



- We are building a flask application which needs HTML pages stored in the Template folder and python script app.py for scripting
- RF.joblib is our saved model. Further we will use this model for flask integration.
- Training folder contains a model training file.

Milestone 1: Data Collection & Preparation

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

Activity 1: Collect the Dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc. In this project we have used .csv data. This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Dataset: [LINK](#)

As the dataset is downloaded. Let us read and understand the data properly with the help of some visualisation techniques and some analysing techniques.

Note: There are a number of techniques for understanding the data. But here we have used some of it. In an additional way, you can use multiple techniques.

Activity 1.1: Importing the libraries

Import the necessary libraries as shown in the image.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import GridSearchCV
import warnings
warnings.filterwarnings('ignore')
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix
```

Activity 1.2: Read the Data set

Our dataset format might be in .csv, excel files, .txt, .json, etc. We can read the dataset with the help of pandas. In pandas we have a function called `read_csv()` to read the dataset. As a parameter we have to give the directory of the csv file.

```
df=pd.read_csv(r"C:\work_files\Data_sets\IoTProcessed_Data.csv")
df.head(5)
```

date	tempreature	humidity	water_level	N	P	K	Fan_actuator_OFF	Fan_actuator_ON	Watering_plant_pump_OFF	Watering_plant_pump_ON	Water_pump_actu
2024-02-08 16:10:00	41	63	100	255	255	255	0.0	1.0	1.0	0.0	
2024-02-08 16:15:00	41	59	100	255	255	255	0.0	1.0	1.0	0.0	
2024-02-08 16:20:00	41	62	100	255	255	255	0.0	1.0	1.0	0.0	
2024-02-08 16:05:00	40	60	100	255	255	255	0.0	1.0	1.0	0.0	
2024-02-08 16:00:00	39	61	100	255	255	255	0.0	1.0	1.0	0.0	

Activity 2: Data Preparation

As we have understood how the data is, let's pre-process the collected data.

The download data set is not suitable for training the machine learning model as it might have so much randomness and noise .So we need to clean the dataset properly in order to fetch good results.

This activity includes the following steps.

- Handling missing values
- Handling categorical data
- Handling Outliers

Note: These are the general steps of pre-processing the data before using it for machine learning. Depending on the condition of your dataset, you may or may not have to go through all these steps.

Activity 2.1: Handling Missing Values

Let's find the shape of our dataset first. To find the shape of our data, the `df.shape` method is used. To find the data type, `df.info()` function is used.

```
df.shape
```

```
(37922, 13)
```

Above Figure Describes the Shape of the Dataset i.e, there are 32537 rows and 15 columns including the Target column as well.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 37922 entries, 0 to 37921
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                  37920 non-null  object
1   temprature            37922 non-null  int64
2   humidity              37922 non-null  int64
3   water_level           37922 non-null  int64
4   N                     37922 non-null  int64
5   P                     37922 non-null  int64
6   K                     37922 non-null  int64
7   Fan_actuator_OFF      37922 non-null  float64
8   Fan_actuator_ON       37922 non-null  float64
9   Watering_plant_pump_OFF 37922 non-null  float64
10  Watering_plant_pump_ON  37922 non-null  float64
11  Water_pump_actuator_OFF 37922 non-null  float64
12  Water_pump_actuator_ON  37922 non-null  float64
dtypes: float64(6), int64(6), object(1)
memory usage: 3.8+ MB
```

`df.info()` provides the information about the column's datatype and provides the count of non-null values in the column is concerned.

Dataset do not have any missing values.

For checking the null values, `df.isnull()` function is used. To sum those null values, we use `.sum()` function. From the below image we found that there no null values present in our dataset:

```
df.isnull().sum()
```

```
date                2
tempreature         0
humidity            0
water_level        0
N                  0
P                  0
K                  0
Fan_actuator_OFF   0
Fan_actuator_ON    0
Watering_plant_pump_OFF 0
Watering_plant_pump_ON 0
Water_pump_actuator_OFF 0
Water_pump_actuator_ON 0
dtype: int64
```

From the above figure we can observe that there are no null values present in the dataset except Date Column.

Activity 2.2: Handling Categorical Values:

There are multiple categorical columns present in the dataset, they are Work class, Sex, Race, Education, Occupation, Relationship, Marital Status and native country.

As we know there are no missing values/ null values present in the dataset. We need to know their number of categories present in each column with their counts.

There are several operations to find different insights using categorical values. Some of the functions are **value_counts**, **replacement of values**.

```
df["Water_pump_actuator"].value_counts()
```

```
Water_pump_actuator
0    29357
1     8565
```

```
df["Watering_plant_pump"].value_counts()
```

```
Watering_plant_pump
0    28900
1     9022
```

Considering only one column for Watering_plant_pump and its actions as ON and OFF as a single column. In the same way we need to consider a single column for Fan_actuator and Water_pump_actuator, simultaneously drop the rest of the columns related to the changed columns and change the type of modified column into Int from float.

```
df['Water_pump_actuator']=df['Water_pump_actuator_ON']

df['Watering_plant_pump']=df['Watering_plant_pump_ON']

df['Fan_actuator']=df['Fan_actuator_ON']

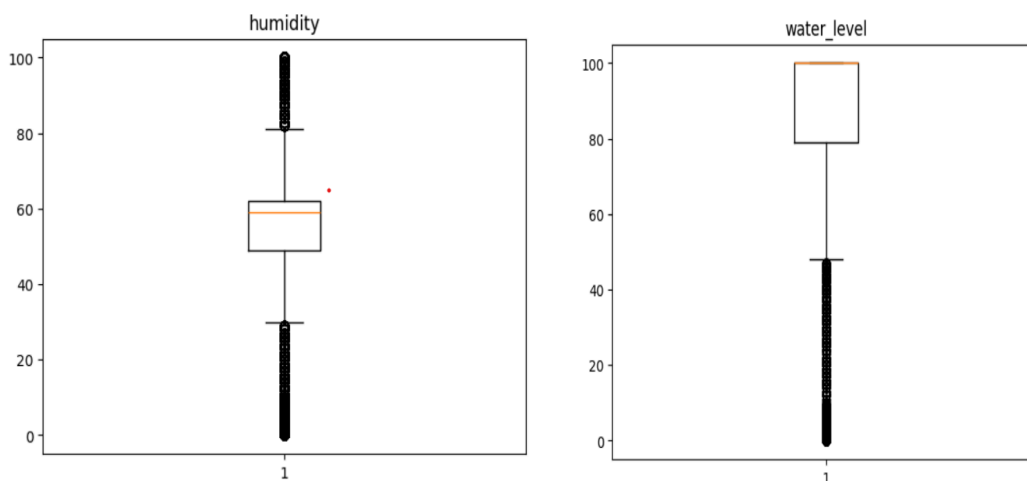
df.drop(['Fan_actuator_ON','Watering_plant_pump_ON','Water_pump_actuator_ON'],axis=1,inplace=True)

df["Watering_plant_pump"]=df["Watering_plant_pump"].astype("int")
df["Fan_actuator"]=df["Fan_actuator"].astype("int")
df["Water_pump_actuator"]=df["Water_pump_actuator"].astype("int")
```

There are no Categorical columns present in the data so there are no columns to encode .

Activity 2.3: Treating Outliers:

Outliers are abnormal data that are away from the range of the distribution of the data of each column in the data. Here we have the box plot to find whether the Outliers are present or not but we cannot know how many Outliers present in the Data



There are outliers present in the data, we need to Handle the outliers for the various columns present in the data. For handling the outliers, we need to find the upper and lower limits. The data beyond these limits are treated as outliers. These outliers are treated using the Winsorizer and Auxiliary capper method.

```
q1=df["water_level"].quantile(0.25)
q3=df["water_level"].quantile(0.75)
iqr1=q3-q1
upper_limit_w=q3+(1.5*iqr1)
lower_limit_w=q1-(1.5*iqr1)
```

Below figure shows the process of handling outliers using the Winsorizer Technique.

```
win1=Winsorizer(capping_method='iqr',tail="both", fold=1.5)
win.fit_transform(df[["water_level"]])
```

Milestone 2: Exploratory Data Analysis

Activity 1: Descriptive Analysis

Descriptive analysis involves examining fundamental characteristics of data using statistical methods. It provides insights into the mean, standard deviation, minimum, maximum, and percentile values of continuous features.

```
df.describe()
```

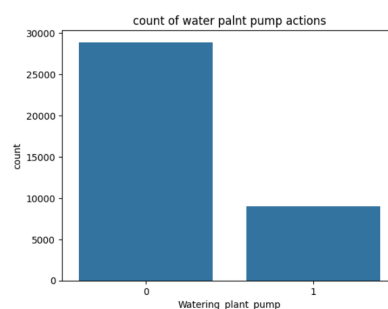
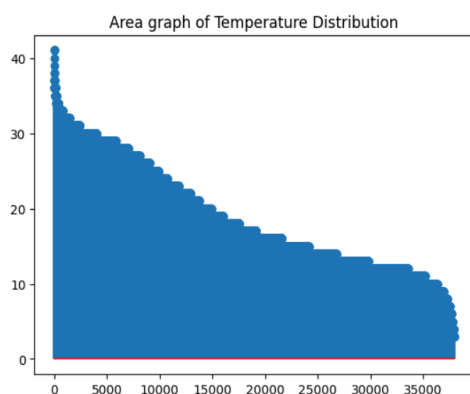
	tempreature	humidity	water_level	N	P	K	Water_pump_actuator	Watering_plant_pump	Fan_actuator
count	37922.000000	37922.000000	37922.000000	37922.000000	37922.000000	37922.000000	37922.000000	37922.000000	37922.000000
mean	18.760113	50.329308	79.401587	231.963610	234.123332	225.171879	0.225858	0.237909	0.310137
std	7.143422	22.995645	36.824610	34.718121	31.452138	44.877635	0.418152	0.425809	0.462555
min	3.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	13.000000	49.000000	79.000000	185.000000	190.000000	160.000000	0.000000	0.000000	0.000000
50%	17.000000	59.000000	100.000000	255.000000	255.000000	255.000000	0.000000	0.000000	0.000000
75%	25.000000	62.000000	100.000000	255.000000	255.000000	255.000000	0.000000	0.000000	1.000000
max	41.000000	100.000000	100.000000	255.000000	255.000000	259.000000	1.000000	1.000000	1.000000

Activity 2: Visual analysis

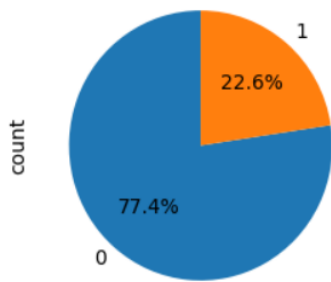
Visual analysis is the process of using visual representations, such as charts, plots, and graphs, to explore and understand data. It is a way to quickly identify patterns, trends, and outliers in the data, which can help to gain insights and make informed decisions.

Activity 2.1: univariate analysis

In simple words, univariate analysis is understanding the data with a single feature. Here we have displayed a histogram, Pie plot, and Horizontal bar plot.

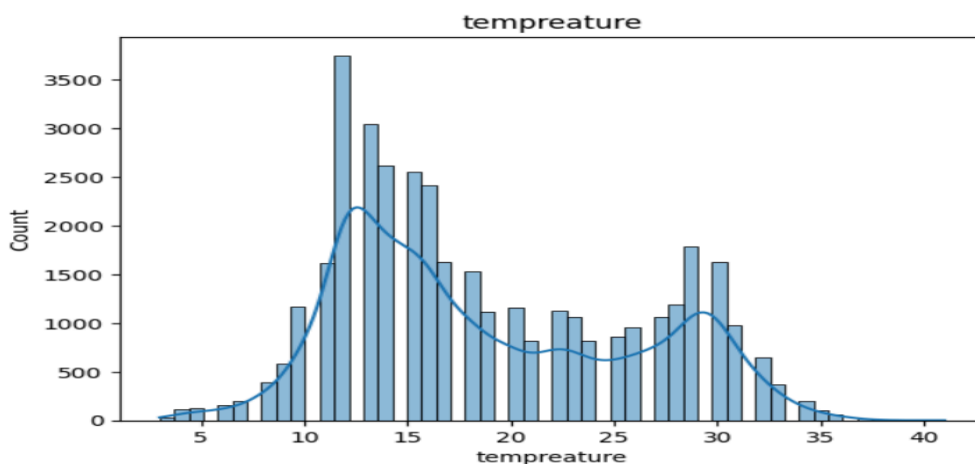


Temperature gradually decreases
From 35 to 15 degrees.



Percentage of output categories.
Water plant pump
Actions (ON & OFF).

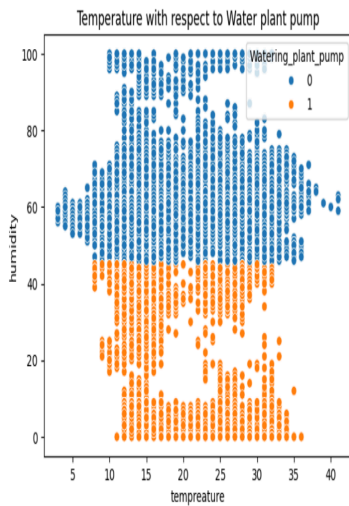
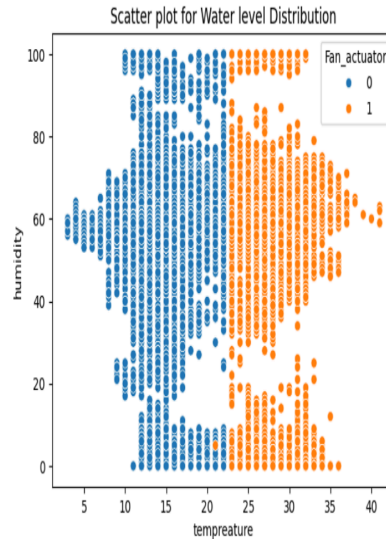
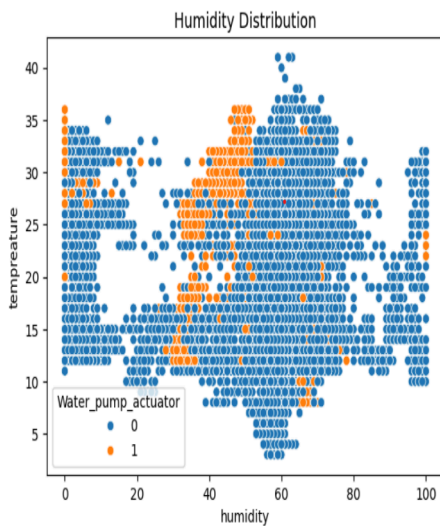
The area plot describes the distribution of Temperature in the dataset and a pie plot describes the percentage of actions by the water pump actuator. A Count plot represents the water Plant pump action of on and OFF positions in count.



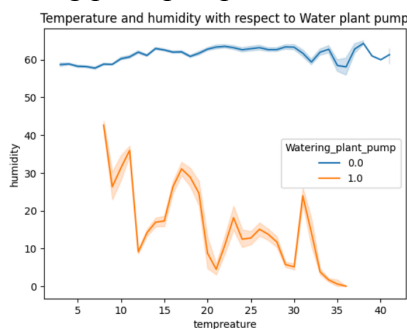
Above Horizontal histogram represents the educational Background of adults.

Activity 2.2: Bivariate analysis

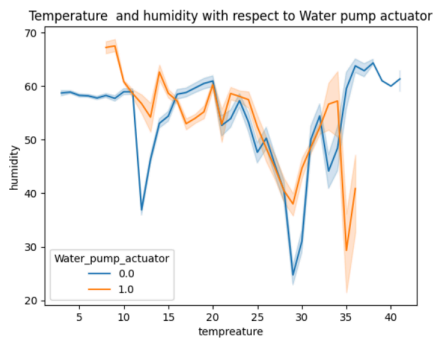
Bivariate analysis is a statistical method that involves the analysis of two variables to determine the empirical relationship between them. Here we have used Scatter plots for finding the insights in each scatter plot.



The above scatter plots represent the actions of the Water pump actuator, Fan actuator, and Watering plant pump within different ranges of temperature and Humidity.



Humidity remains constant when Temperature is constant. Humidity constantly decreases by Increasing in temperature when the water Plant pump in the ON position.

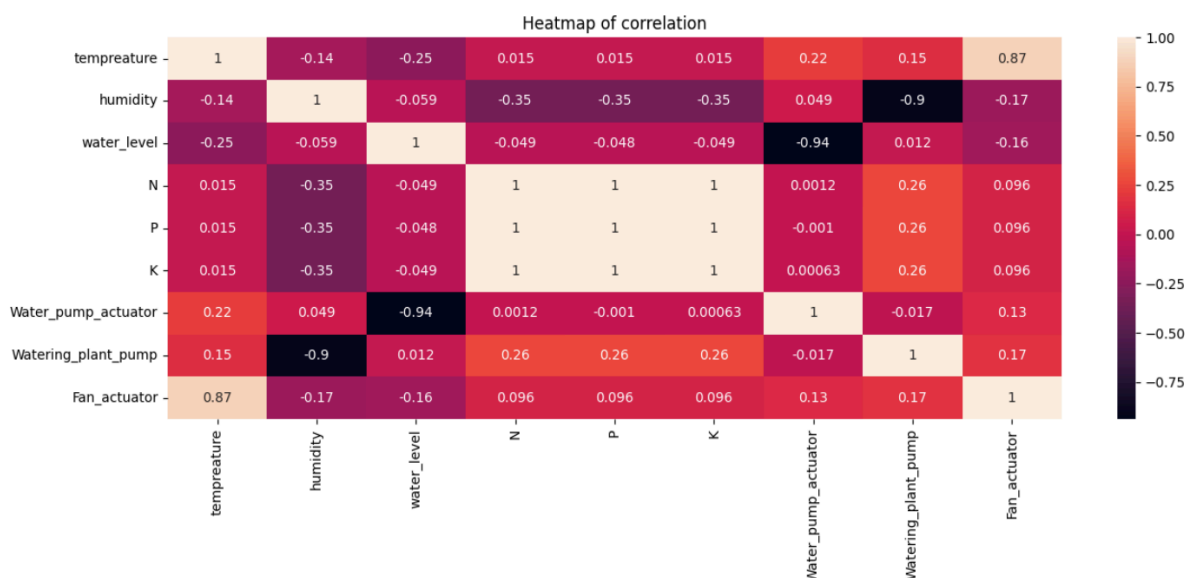


Small Change is observed in humidity along with the change in temperature when the water pump actuator is in off position.

Gradual drop of humidity with the change in temperature when the water pump actuator is in on position.

Activity 2.3: Multi-variate analysis

Multi-variate analysis is a statistical method that involves the analysis of more than 2 variables to determine the empirical relationship among them. Here we have a heatmap representing the correlation among the variables in the Data.



Humidity is non correlated to Water_plamt_pump

Water_level in non correlated to Water_pump_actuator

Activity 3: Balancing

Data is imbalanced since the category for the target category column counts having a major difference between the categories so we need to make the data balanced so we use SMOTE technique to balance the data.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
x_sample, y_sample = smote.fit_resample(x, y)
```

Activity 4: Splitting data into train and test

Now let's split the Dataset into train and test sets. First, split the dataset into x and y and then split the data set. "x" represents the whole data columns other than the target column, "y" represents the Target column in the dataset. We need to build the model by giving the training to the model and making the predictions on the test data. so we need to divide the whole dataset into training and testing data.

For splitting training and testing data we are using the `train_test_split()` function from `sklearn`. As parameters, we are passing x, y, test_size, random_state.

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x_sample,y_sample,test_size=0.2,random_state=1)
```

Milestone 3: Model Building

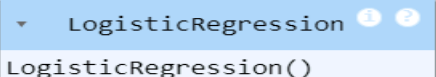
Activity 1: Training and testing the models using multiple algorithms

Now our data is cleaned and it's time to build the model. We can train our data on different algorithms. For this project, we are applying classification algorithms. The best model is saved based on its performance.

Activity 2.1 Logistic Regression:

Logistic Regression is the baseline model where we import the Logistic Regression from the `sklearn` library and import the `linear_model` and assign the model to the variable `model1` and make the data fit and process the data with respect to the training data.

```
from sklearn.linear_model import LogisticRegression
model1=LogisticRegression()
model1.fit(x_train,y_train)
```



LogisticRegression

LogisticRegression()

```
y_pred_train1= model1.predict(x_train)
y_pred_test1=model1.predict(x_test)
```

```
from sklearn.metrics import accuracy_score
train_acc1= accuracy_score(y_train,y_pred_train1)
test_acc1=accuracy_score(y_test,y_pred_test1)
```

```
print("train Accuracy",train_acc1)
print("test Accuracy",test_acc1)
```

```
train Accuracy 0.9853100849460306
test Accuracy 0.9846717193221494
```

```
from sklearn.model_selection import cross_val_score
print("cross_val_score",cross_val_score(model1,x_sample,y_sample, cv=5).mean())
cross_val_score 0.978029209078964
```

```
confusion_matrix(y_train,y_pred_train1)
```

```
array([[23158,   310],
       [   380, 23123]], dtype=int64)
```

We make the predictions on the train and test data using the predict function and assigning the predicted values to the y_pred_train and y_pred_test

We are calculating the accuracy scores on how the model is working on the data and we use cross-validations to reduce the Bias and trade condition to the dataset. Actually we are able to divide the dataset based on the chosen test size. If CV=4. Then we trained the model with 75% of data and we tested with 25% of the data. If CV=5 then we are training the model with 80% of data and testing the model with 20% of the data.

Activity 2.2: Decision Tree Classifier:

Decision Tree Classifier and regression algorithm is initialised and the training data is passed to the model and assigned the variable as model2 with the .fit() function. Test data is predicted with model2.predict() function and saved in a new variable. For evaluating the model, accuracy is calculated

```
from sklearn.tree import DecisionTreeClassifier
model2=DecisionTreeClassifier(criterion='gini',max_depth= 5,splitter= 'best')
model2.fit(x_train,y_train)
```

```
▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=5)
```

```
y_pred_train2=model2.predict(x_train)
y_pred_test2=model2.predict(x_test)
```

```
train_acc2=accuracy_score(y_train,y_pred_train2)
test_acc2=accuracy_score(y_test,y_pred_test2)
print("train accuracy", train_acc2)
print("test accuracy",test_acc2)
```

```
train_accuracy 0.9944008004939218
test accuracy 0.9942944733032445
```

```
precision=precision_score(y_train,y_pred_train2)
recall=recall_score(y_train,y_pred_train2)
f1=f1_score(y_train,y_pred_train2)
print("precision",precision)
print("recall",recall)
print("f1_score",f1)
```

```
precision 0.9913734776725305
recall 0.9974896821682339
f1_score 0.9944221755636148
```

```
confusion_matrix(y_train,y_pred_train2)
```

```
array([[23264, 204],
       [ 59, 23444]], dtype=int64)
```

At the end, we need to observe the train accuracy and test accuracy with a cross-validation score. There is a constraint that the difference between train and test accuracy should not be more than 5% and the difference of test accuracy and cross-validation should not be more than 5% the best model is fixed by this scenario.

Activity 2.2 : Random-Forest Classifier

Random Forest algorithm is the classification and regressions algorithm initialised and training data is passed to the model and assigned to the variable as model3 with the .fit() function. Test data is predicted with model3.predict() function and saved in a new variable. For evaluating the model accuracy is calculated. For the best obtaining of accuracy, we use hyperparameter tuning to tune the model with the best hyperparameters using the Grid Search CV by choosing the best params we can able to get the best accuracy this method is known as Hyperparameter Tuning.

```
from sklearn.ensemble import RandomForestClassifier
model3=RandomForestClassifier()
model3.fit(x_train,y_train)

RandomForestClassifier()

y_pred_train3=model3.predict(x_train)
y_pred_test3=model3.predict(x_test)

train_acc3=accuracy_score(y_train,y_pred_train3)
test_acc3=accuracy_score(y_test,y_pred_test3)
print("train accuracy",train_acc3)
print("test accuracy", test_acc3)
print("cross_val_score",cross_val_score(model3,x_train,y_train, cv=5).mean())

train accuracy 0.9981690830512444
test accuracy 0.996678872519799
cross_val_score 0.996657502104941
```

Milestone 4: Performance Testing

Under performance Testing, we need to test the model's accuracy with different testing Metrics like Precision, Recall, and F1_score. Below are the performance Metrics of the final fixed model

```
precision=precision_score(y_train,y_pred_train3)
recall=recall_score(y_train,y_pred_train3)
f1=f1_score(y_train,y_pred_train3)
print("precision",precision)
print("recall",recall)
print("f1 score",f1)

precision 0.9974508221098696
recall 0.9988937582436285
f1 score 0.998171768707483

print(confusion_matrix(y_train,y_pred_train3))

[[23408    60]
 [   26 23477]]
```

Milestone 5: Model Deployment

Activity 1: Save and load the best model

Saving the best model after comparing its performance using different evaluation metrics means selecting the model with the highest performance. This can be useful in avoiding the need to retrain the model every time it is needed and also to be able to use it in the future.

```
import joblib
joblib.dump(model3, open("RF.joblib", "wb"))
```

Activity 2 : Test the model

Let's test the model first in the Python notebook itself. As we have 7 features in this model, let's check the output by giving all the inputs.

```
print(model3.predict([[36,49,0,255,255,255,0,1]]))
```

[1]

```
print(model3.predict([[47,52,0,225,255,0,1,0]]))
```

[1]

```
print(model3.predict([[41,63,100,255,255,255,0,1]]))
```

[0]

```
print(model3.predict([[46,89,100,215,215,155,1,1]]))
```

[0]

The predicted values result the same as actual values.

Activity 3: Integrate with Web Framework

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building server-side script
- Run the web application

Activity 3.1: Building Html Pages:

For this project create two HTML files namely

- index.html
- predict.html
- result.html

and save them in the templates folder.

Activity 3.2: Build Python code:

Import the libraries

```
import numpy as np
import joblib
import pandas as pd
from flask import Flask, request, render_template
```

Load the saved model. Importing the Flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (__name__) as argument.

```
app = Flask(__name__)
try:
    model = joblib.load('RF.joblib')
except Exception as e:
    print(f"Error loading model: {e}")
```

We render index.html for the displaying the web application , similarly we render the predict.html for the user input values of the forms to predict the income. Simultaneously we render result .html to display the result of the prediction value.

Render Index.html:

```
@app.route('/')
def home():
    """Render the home page."""
    return render_template('index.html')
```

Here we will be using a declared constructor to route to the HTML page which we have created earlier.

In the above example, '/' URL is bound with the index.html function. Hence, when the home page of the web server is opened in the browser, the html page will be rendered.

Render Predict.html:

```
@app.route('/predict')
def predict():
    """Render the prediction page."""
    return render_template('predict.html')
```

In the predict.html where we provide the user inputs in the form for the prediction of income

Whenever you enter the values from the html page the values can be retrieved using POST and GET Methods.

Retrieving the value from UI:

```
@app.route('/submit', methods=["POST"])
def submit():
    """Handle form submission and make predictions."""
    try:
        # Reading the inputs given by the user
        input_feature = [x for x in request.form.values()]
        input_feature = [np.array(input_feature)]

        # Define column names
        names = ['tempreature', 'humidity', 'water_level', 'N', 'P', 'K',
                'Watering_plant_pump', 'Fan_actuator']

        # Create a DataFrame
        data = pd.DataFrame(input_feature, columns=names)

        print(data)

        # Predictions using the loaded model file
        prediction = model.predict(data)
        print(prediction)

        # Determine the result message based on the prediction value
        if prediction[0] == 0:
            result = "Water pump actuator OFF"
            prediction = "OFF"
        else:
            result = "Water pump actuator ON"
            prediction = "ON"

        return render_template("Result.html", result=result, prediction=prediction)
    except Exception as e:
        # Handle exceptions and print error for debugging
        print(f"Error during prediction: {e}")
        return render_template("Result.html", result="An error occurred. Please try again.")
```

Here we are routing our app to a conditional statement. This will retrieve all the values from the HTML page using a Post request. That is stored in an array. This array is passed to the model.predict() function. This function returns the prediction. This prediction value will be rendered to the text that we have mentioned in the submit.html page earlier.

Main Function:

```
if __name__ == "__main__":  
    app.run(debug=True, port=4000) # Running the app with debug enabled
```

Activity 3.3: Run the web application

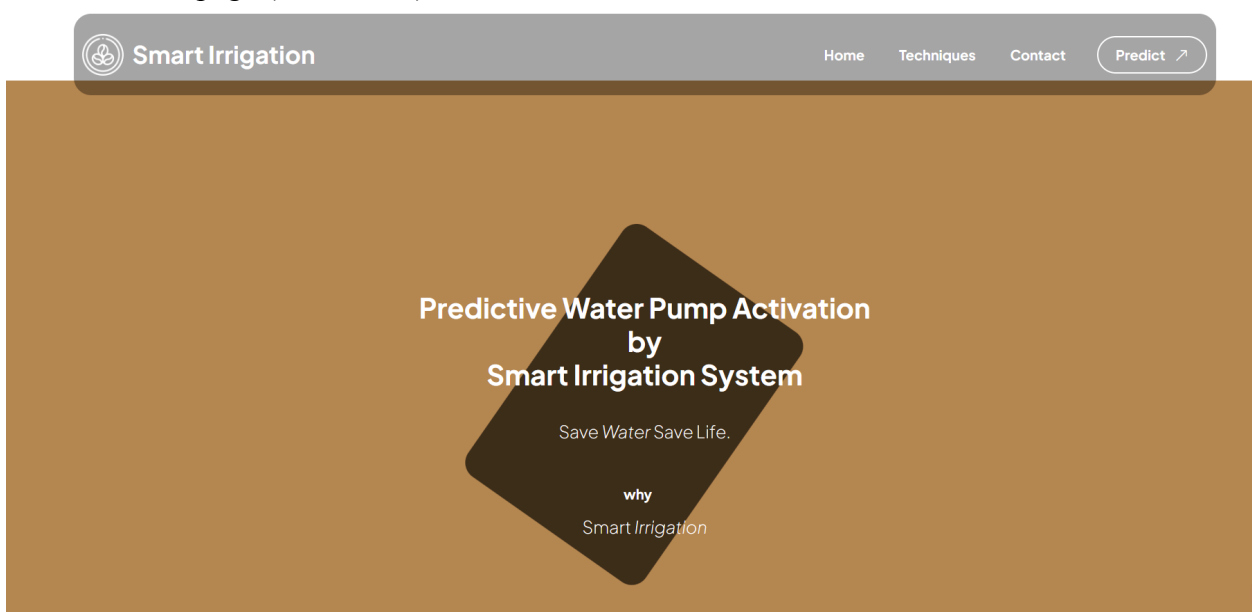
- Open vs code application in the search menu.
- Navigate to the folder where your flask folder of your files exists.
- Click on the view button in the vs code nav bar and click on the terminal option in the dropdown menu.
- Now type “app.py” command
- You will have a link displayed in the terminal as “<http://127.0.0.1:4000>”.
- Double-click on the link then you will be navigated to the web application.
- Click on the predict button in the nav bar, enter the inputs, click on the predict button, and see the result/prediction in the result.html.

```
PS C:\work_files\projects\IOT_Water\Smart_Irrigation> python app.py  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:4000  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 974-677-858
```

Now, Go the web browser and write the localhost URL (<http://127.0.0.1:4000>) to get the below results

Results:

- a. Index page (Index.html)



EQUIPMENTS OF

SMART IRRIGATION

IoT Irrigation

ardino

Dedicated code is written in the Embedded C in the Arduino chip to function the Water pump



Sprinklers

IoT/Ardino

IoT/Ardino Sprinklers works on the moisture and the temperature of the soil



Garden Sprinklers

MC

Garden sprinklers works by using Micro controllers. These controllers contains code of c used for functioning of water pump



Moisture

sensors

Sensors find the range of moisture. Based on it water is pumped to field.



Contact Details

Say Hello

Contact

Name

Jackson

Email Address

Jack@gmail.com

How can we help?

Message

Send Message

Lover Point

Bandra Fort Trail, Mount Mary,

Bandra West, Mumbai, Maharashtra

400050, India

4.4 ★★★★★

54 reviews

View larger map

Basilica of Our Lady of the Mount

बसिलिका ऑफ द लेडी ऑफ द माउंट

बसिलिका ऑफ द लेडी ऑफ द माउंट

Bandra Fort

बंदरा फोर्ट

बंदरा फोर्ट

Mumbai Maiden

मुंबई मेयन

मुंबई मेयन

BWSL Promenade

बि.व्ही.सी.ली. प्रमोनाद

बि.व्ही.सी.ली. प्रमोनाद

World Sea Link Bridge

वर्ल्ड सी लिंक ब्रिज

वर्ल्ड सी लिंक ब्रिज

Google

Keyboard shortcuts

Map data ©2024

Imagery ©2024

Airbus, CNES / Airbus, Maxar Technologies

Terms

Report a map error

If you click on the predict on the nav bar we can able to navigate to predict.html below figure represents the UI of predict.html page

b) Prediction page (Predict.html)

www.smartinternz.com

Page 19 of 20

Smart Irrigation

Home

Temperature

Type value of Temperature (0-50)

Humidity

Type value of Humidity (0-100)

Water Level

Type the level of Water (0-100)

Potassium Level

Type the level of Potassium (0-255)

Nitrogen Level

Type the level of Nitrogen (0-255)

Phosphorous Level

Type the level of Phosphorous (0-255)

Watering Plant Pump

Water Pump Plant

Fan Actuator

Fan

Predict

By providing the inputs by the user and click on the predict button you will be navigating to the result.html it displays the result. Based on the result you can navigate to the home and click on the investments in the nav bar of index.html then you can have an info about different mode of investments.

Result.html:

Smart Irrigation

Home

Prediction Results

Status of Water Pump actuator is OFF